AGENTES DE IA · 2026-2

PonyArchive

Biblioteca inteligente de **My Little Pony: Friendship Is Magic**
con agente RAG, LangGraph y modelo local

Integrante: **Mariana Lemus Esguerra** · Trabajo individual

Problema o necesidad

Un LLM responde **de memoria**: suena seguro incluso cuando no sabe.

Fundamentar

Cada respuesta debe salir de los documentos del corpus y citar su fuente — no de la memoria paramétrica del modelo.

Rechazar

Fuera del corpus, decir «no lo sé» con la frase literal exigida, en vez de inventar algo plausible.

> «No tengo suficiente información en mi base de conocimiento para responder esa pregunta.»

Caso adicional: **conocimiento de contexto del aula** — datos que constan en una nota propia, no en ningún documento público, deben responderse citando esa nota.

Arquitectura

Frontend
React + Vite + TS
chat · documentos · journal

⇄

Backend FastAPI
21 endpoints · JWT
grafo LangGraph

⇄

ChromaDB + SQLite
819 fragmentos
usuarios y conversaciones

⇄

Ollama (local)
qwen3.5:4b
RTX 4070 · 67 tok/s

Sin nube de pago

Todo local o free tier: presupuesto **0 USD**.

Privacidad

Ninguna pregunta ni documento sale del equipo.

Docker

`docker compose up -d` levanta backend (8010) y frontend (8081).

Tecnologías usadas

CAPA	TECNOLOGÍA	POR QUÉ
Backend	Python 3.14 · FastAPI · Uvicorn	Encaja con LangChain; OpenAPI en `/docs`
Agente	LangChain + LangGraph	Grafo de estados en código, sin constructores visuales
LLM	Ollama — `qwen3.5:4b`	Local, gratuito, 67 tok/s en RTX 4070
Embeddings	`multilingual-e5-small` (384 d)	Multilingüe: rinde en español
Vectorial	ChromaDB persistente	Embebida, sin servidor externo
Frontend	React + Vite + TypeScript	Ligero; build con `tsc` + vite sin errores
Datos	Pony API · CC BY-SA 3.0	243 episodios · 555 personajes

Secretos en `.env` (nunca en el repo) · código bajo licencia MIT · dependencias fijadas.

Agente con LangChain y LangGraph

validate_intent
dominio · catálogo · fuera de dominio

→

retrieve
Chroma filtrada por usuario

→

evaluate_sufficiency
¿hay evidencia?

generate
prompt restringido + citas [n]

→

validate_answer
entidades en el contexto

→

persist
historial del usuario

Rutas condicionales: sin evidencia → **reject** (frase literal) · consultas de catálogo → SQL directo sin LLM.

```
g = StateGraph(AgentState)
g.add_node("retrieve", retrieve)
g.add_conditional_edges("evaluate_sufficiency",
    lambda s: "generate" if s["sufficient"] else "reject")
```

Base vectorial y carga de documentos

Cargar
Pony API · semillas · .txt/.md del usuario

→

Trocear
fragmentos con solapamiento

→

Embeber
e5-small · "passage: …"

→

Indexar
Chroma + metadatos

Metadatos por fragmento

`doc_type`, `title`, `user_id`, `themes`, `chunk_index` → cada fuente reconstruye su documento original.

Idempotente y aislado

Deduplicación por `content_hash` (por usuario); las cargas de un usuario no se ven desde otro.

Consulta: `TOP_K=6` · umbral `0.62` · bonificación +0,05 a los documentos propios recién subidos.

Seguridad y usuarios

Autenticación

- Registro / login / refresh / logout
- Contraseñas con hash · JWT firmado con `SECRET_KEY`
- Sin token en endpoints de negocio → **401**

Aislamiento de datos

- Historial por usuario (caso B04)
- Documentos propios vs. compartidos (T10/T10b)
- Cada búsqueda filtra por visibilidad

Configuración

- CORS con lista explícita de orígenes
- `.env` fuera del repo · `.env.example` documentado
- Contenedor sin privilegios

DEC-05 · Keycloak

Se eligió Keycloak; opera en modo `local` (JWT propio) con `AUTH_PROVIDER=keycloak` listo para activarse. El enunciado permite implementación propia.

Demostración

✓ Respondido con fuentes

«¿Qué ocurre en Lesson Zero?» → respuesta con **6 fuentes** y similitudes.

✗ Rechazado

«¿Cuánto sulfato de cobre…?» → frase literal de rechazo.

📄 Doc nuevo

Se sube un .txt y la siguiente pregunta responde desde ese archivo.

📚 Contexto del curso

«¿A Duvan Gomez le pega la esposa?» → sí, con la nota como fuente (sim 0,904).

👥 Aislamiento

Dos usuarios: cada uno solo ve y consulta sus documentos.

🧪 Pruebas

12/12 smoke E2E Docker OK

Recorrido en vivo: login → chat con fuentes → rechazo → carga de documento → grafo en el código → despliegue.

Conclusiones

1. Requisitos mínimos cumplidos: backend + frontend, auth, agente con LangGraph, **modelo local**, base vectorial con carga de documentos, respuestas fundamentadas y **rechazo explícito**.

2. 7 estrategias anti-alucinación (exigidas ≥2), verificadas con **12 pruebas automatizadas** y un E2E completo sobre Docker.

3. LangGraph hace auditables las decisiones del agente: cada ruta es una arista en código, sin constructores visuales.

4. Un modelo de 4B basta cuando el RAG hace el trabajo pesado: la calidad depende de la recuperación y del prompt, no del tamaño del modelo.

5. Coste 0 USD: todo local o free tier, con el túnel al Ollama del portátil como solución al modelo en la nube.

Repo: `github.com/navyri/PonyArchive` · Documento técnico con las 14 secciones · Licencias: código MIT · datos CC BY-SA 3.0 (Pony Archive).